



UNIVERSIDAD
NACIONAL
DE COLOMBIA



Sesión 2: Funciones Puras e Inmutabilidad

Semillero en Ciencias de la Computación

Tomás Rodríguez Santiago López

Mayo 8, 2026

1. Conceptos Teóricos
 - Funciones Puras
 - Inmutabilidad
 - Estado mutable compartido
2. Mutabilidad en algunos paradigmas de programación
3. Aplicación en Clojure
 - Colecciones Inmutables por Defecto
 - Modificando Datos
4. Ejercicios Prácticos
 - Función Pura: IVA
 - Estado inmutable: Usuario



1. Conceptos Teóricos
 - Funciones Puras
 - Inmutabilidad
 - Estado mutable compartido

Recordemos la siguiente definición:

Definición:

Una función se dice *pura* si cumple con:

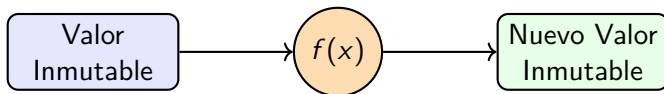
1. **Retorna un solo valor.**
2. **El resultado depende únicamente de sus argumentos.**
3. **No muta ningún valor existente** (cero efectos secundarios).

Si las funciones puras componen el motor de los programas funcionales, los valores inmutables son su **combustible, aceite y gases**.

Definición:

Una vez que un dato es creado, **no puede ser cambiado**.

En lugar de modificar un valor, las funciones puras **hacen copias de los datos y las pasan como resultado**.



Los conceptos de función pura e inmutabilidad son tan expresivos que la programación funcional en sí misma se puede definir en únicamente estos términos:

Definición

Programación funcional es programar usando *funciones puras* que manipulan valores *inmutables*.

Suponga que está planeando un eurotrip con sus amigos el cual luce así:



Figura: Ruta de ciudades

Manjeamos nuestro itinerario así:

```
List<String> planA = new ArrayList<>();  
planA.add("Paris");  
planA.add("Berlin");  
planA.add("Krak w");  
System.out.println("Plan A: " + planA);
```

Lo cual produce:

Plan A: [Paris, Berlin, Kraków]

Sin embargo, uno de nuestros amigos es un gran fan de Mozart el cual insiste en visitar Vienna antes de ir a Kraków.

Ejercicio

Proponga una función para implementar este cambio en el itinerario.

Considere esta solución a la modificación del itinerario

```
// Tipo de retorno inofensivo...  
static List<String> replan(List<String> plan, String  
newCity, String beforeCity) {  
    int newCityIndex = plan.indexOf(beforeCity);  
    plan.add(newCityIndex, newCity);  
    return plan;  
}
```

1. ¿Qué está haciendo la función?
2. ¿Qué tipo de dato está devolviendo?
3. ¿Es una función pura?

Considere esta solución a la modificación del itinerario

```
// Tipo de retorno inofensivo...  
static List<String> replan(List<String> plan, String  
newCity, String beforeCity) {  
    int newCityIndex = plan.indexOf(beforeCity);  
    plan.add(newCityIndex, newCity);  
    return plan;  
}
```

¿Qué pasa aquí?

Aunque la función promete devolver un plan nuevo, en realidad muta la lista original en memoria que se le pasó como argumento. Mentir en la firma rompe nuestra intuición y es una de las mayores causas de bugs.

Muchas preguntas surgen al observar estos comportamientos en funciones:

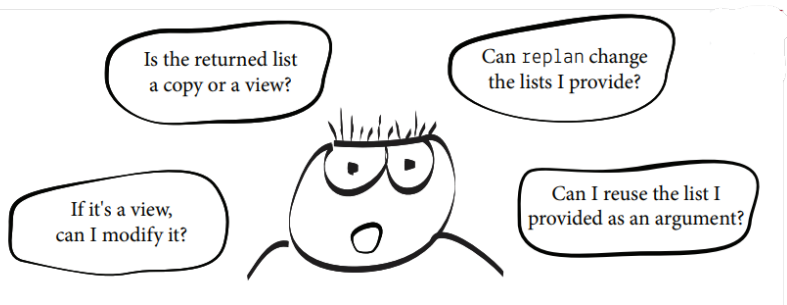


Figura: Los valores mutables son confusos.

Considere esta modificación de replan

```
static List<String> replan(List<String> plan, String
newCity, String beforeCity) {
    int newCityIndex = plan.indexOf(beforeCity);
    // Creamos una copia de la lista original
    List<String> replanned = new ArrayList<>(plan);
    replanned.add(newCityIndex, newCity);
    return replanned;
}
```

1. ¿Es esta función pura?
2. ¿Es esto un ejemplo de programación funcional?

Considere esta modificación de replan

```
static List<String> replan(List<String> plan, String
newCity, String beforeCity) {
    int newCityIndex = plan.indexOf(beforeCity);
    // Creamos una copia de la lista original
    List<String> replanned = new ArrayList<>(plan);
    replanned.add(newCityIndex, newCity);
    return replanned;
}
```

Es pura!!

Definición:

Una función se dice *pura* si cumple con:

1. **Retorna un solo valor.**
2. **El resultado depende únicamente de sus argumentos.**
3. **No muta ningún valor existente** (cero efectos secundarios).

Considere esta modificación de replan

```
static List<String> replan(List<String> plan, String
newCity, String beforeCity) {
    int newIndex = plan.indexOf(beforeCity);
    // Creamos una copia de la lista original
    List<String> replanned = new ArrayList<>(plan);
    replanned.add(newIndex, newCity);
    return replanned;
}
```

No es programación funcional. ¿Por qué?

Definición

Programación funcional es programar usando *funciones puras* que manipulan valores *inmutables*.

Considere esta modificación de replan

```
static List<String> replan(List<String> plan, String
newCity, String beforeCity) {
    int newCityIndex = plan.indexOf(beforeCity);
    // Creamos una copia de la lista original
    List<String> replanned = new ArrayList<>(plan);
    replanned.add(newCityIndex, newCity);
    return replanned;
}
```

Transparencia Referencial

Al no mutar argumentos externos, podemos llamar la función múltiples veces con los mismos argumentos y siempre obtener el mismo resultado.

¿Qué tan complicada puede
ponerse la mutabiliad?

Stay tuned!!

Para ver que tan complicado puede volverse pensar en valores que van cambiando introducimos las siguientes definiciones:

Definición

Un **estado** es una instancia de un valor que está almacenado en un lugar y puede ser consultado por código.

Definición

Cuando un estado puede ser modificado, tenemos un estado mutable.

Definición

Cuando un estado mutable puede ser accedido por diferentes parte del código tenemos un estado mutable compartido.

Veamos el problema de un estado mutable compartido en acción.

Queremos analizar los tiempos de las vueltas de un coche de carreras. Tenemos una lista con los tiempos y necesitamos calcular:

1. El **tiempo total** de la carrera (ignorando la primera vuelta, que es de calentamiento).
2. El **tiempo promedio** de todas las vueltas.

Estado Inicial en Java

```
ArrayList<Double> lapTimes = new ArrayList<>();  
lapTimes.add(31.0); // Vuelta de calentamiento  
lapTimes.add(20.9);  
lapTimes.add(21.1);  
lapTimes.add(21.3);
```

Creamos nuestra función para calcular el tiempo total. Para ignorar la vuelta de calentamiento, simplemente la eliminamos de la lista.

```
static double totalTime(List<Double> lapTimes) {  
    lapTimes.remove(0); // Removemos la vuelta de  
    calentamiento  
  
    double sum = 0;  
    for (double x : lapTimes) {  
        sum += x;  
    }  
    return sum;  
}
```

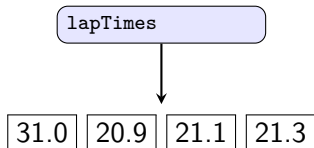
Creamos nuestra función para calcular el tiempo promedio.

```
static double avgTime(List<Double> lapTimes) {  
    double time = totalTime(lapTimes);  
    int laps = lapTimes.size();  
    return time / laps;  
}
```

El Impacto de la Mutabilidad: Resultados Incoherentes

Definimos la lista y corremos las funciones en orden:

```
System.out.println(totalTime(  
    lapTimes));  
// -> Resultado: 63,3  
  
System.out.println(avgTime(lapTimes)  
    );  
// -> Resultado: 21.2 (<-- Es esto  
    correcto?)
```



Ejecución secuencial

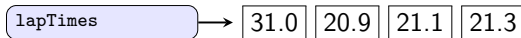
Problema Oculto

`totalTime` tiene un **efecto secundario**: está mutando la lista original que se le pasó como argumento.

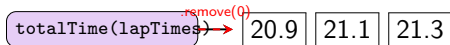
¿Por qué falla? Rastreando el Estado Mutable Compartido



Estado 1: Inicio



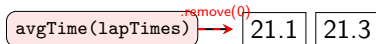
Estado 2: Ejecución de `totalTime`



`totalTime` lee esta lista.

Resultado: 63.3

Estado 3: Ejecución de `avgTime`



`avgTime` lee la lista incompleta.

Resultado: 21.2 !!

Para evitar el problema, debemos tratar la lista de entrada como **immutable**. Si necesitamos modificarla para nuestro cálculo, creamos una copia local.

```
static double totalTime(List<Double> lapTimes) {  
    List<Double> withoutWarmup = new ArrayList<>(  
        lapTimes);  
  
    withoutWarmup.remove(0);  
  
    double sum = 0;  
    for (double x : withoutWarmup) {  
        sum += x;  
    }  
    return sum;  
}
```


En Clojure, este problema no existe desde el diseño, ya que las estructuras de datos son inmutables por defecto. No necesitamos hacer copias manualmente.

Solución Funcional Pura en Clojure

```
(def lap-times [31.0 20.9 21.1 21.3])  
  
(defn total-time [laps]  
  (reduce + (rest laps)))
```

SALIDA:

```
user=> (total-time lap-times)  
63.3
```

```
user=> lap-times  
[31.0 20.9 21.1 21.3]
```



Figura: Noticia del evento de Knight Capital Group - 1 de agosto de 2012.

Como hemos visto hasta el momento los estado mutables dificultan el trabajo de **predecir el comportamiento del programa**.

Considere las siguientes preguntas:

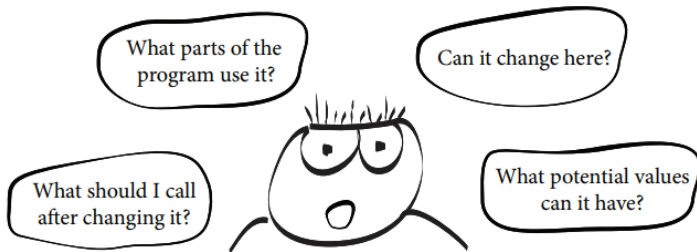


Figura: La mutabilidad causa confusión.

Veamos como se lidia con esto en algunos paradigmas de programación:

2. Mutabilidad en algunos paradigmas de programación

Nuestro enfoque se basó en hacer que nuestra función sea pura.

```
static List<String> replan(List<String> plan,
                           String newCity,
                           String beforeCity) {
    int newCityIndex = plan.indexOf(beforeCity);
    List<String> replanned = new ArrayList<>(plan);
    replanned.add(newCityIndex, newCity);
    return replanned;
}
```

Esto garantiza que no se modifique ningún valor **externo**.

En la programación orientada a objetos probablemente usaríamos la *encapsulación* para asegurar nuestra información de ser mutada de manera indeseada.

Definición

Encapsulación es una técnica que **aísla** un estado mutable, usualmente dentro de un objeto. Este objeto resguarda el estado haciendolo *privado* y asegurandose que solo se interactúe con el usando la interfaz provista por la clase.

Así, la responsabilidad de manipular el estado es restringida a un solo lugar.

En la programación orientada a objetos probablemente usaríamos la *encapsulación* para asegurar nuestra información de ser mutada de manera indeseada.

```
private List<String> plan = new ArrayList<>();  
public void replan(String newCity, String beforeCity  
) {  
    int newCityIndex = plan.indexOf(beforeCity);  
    plan.add(newCityIndex, newCity);  
}  
public void add(String city) {  
    plan.add(city);  
}  
public List<String> getPlan() {
```

Simplemente no hay valores mutables.

Definición

Programación funcional es programar usando *funciones puras* que manipulan valores *inmutables*.

Esto garantiza que una vez un valor es creado, nunca será modificado. Si el programador necesita realizar un cambio, por más mínimo que sea, un nuevo valor necesita ser creado y el antiguo queda intacto.

Veamos como se implementaría *replan* usando **Clojure**:

```
(defn replan [plan new-city before-city]
  (let [[cities-before cities-after]
        (split-with #(not= % before-city) plan)]
    (concat cities-before [new-city] cities-after)))
```

La magia de split-with

Divide la colección en dos partes usando una condición. En este caso, iteramos *mientras la ciudad no sea igual a la ciudad objetivo*. Devuelve:

1. `cities-before`: Los elementos antes del corte.
2. `cities-after`: El resto de la lista original.



3. Aplicación en Clojure

- Colecciones Inmutables por Defecto
- Modificando Datos

Aplicación en Clojure

A diferencia de Java, en Clojure **todas las colecciones base son inmutables por defecto**. No tenemos que preocuparnos por hacer copias manualmente.

- **Vectores** [] : Colecciones indexadas.
- **Listas** () : Colecciones secuenciales.
- **Mapas** {} : Colecciones clave-valor (Diccionarios).

"Modificando" Datos: Creando Copias Eficientes



Clojure utiliza *Structural Sharing* bajo el capó para que “modificar” y crear copias no sea costoso en memoria.

Agregando elementos a un vector con conj

```
user=> (def mi-vector [1 2 3])  
#'user/mi-vector  
  
user=> (conj mi-vector 4)  
[1 2 3 4] ; Retorna un NUEVO vector  
  
user=> mi-vector  
[1 2 3] ; El original permanece intacto
```



Modificando un mapa con assoc

```
user=> (def persona {:nombre "Ana" :edad 25})
#'user/persona

user=> (assoc persona :edad 26)
{:nombre "Ana", :edad 26} ; Retorna un NUEVO mapa

user=> persona
{:nombre "Ana", :edad 25} ; El original es inmutable
```

En Clojure, las funciones de actualización siempre retornan un nuevo valor, respetando la pureza estructural.



4. Ejercicios Prácticos

Función Pura: IVA
Estado inmutable: Usuario

Ejercicios Prácticos

Ejercicio 1: Función Pura para calcular IVA

Sintaxis de *defn*:

Objetivo: Crear una función pura que reciba un precio y devuelva el total con el IVA (19%) incluido.

```
(defn saludar []  
  (println " Hola    al semillero!"))
```

Ejercicio 1: Función Pura para calcular IVA

Objetivo: Crear una función pura que reciba un precio y devuelva el total con el IVA (19%) incluido.

```
;; Nuestra funcion pura
(defn calcular-iva [precio]
  (* precio 1.19))
;; Prueba en el REPL
user=> (calcular-iva 1000)
1190.0
user=> (calcular-iva 500)
595.0
```

Definición:

Una función se dice *pura* si cumple con:

1. **Retorna un solo valor.**
2. **El resultado depende únicamente de sus argumentos.**
3. **No muta ningún valor existente** (cero efectos secundarios).



Objetivo: Definir un mapa de usuario y una función para agregarle puntos **sin mutar** el original.

```
(defn saludar []  
  (println " Hola al semillero!"))
```

Ejercicio 2: Sistema de Puntos



Objetivo: Definir un mapa de usuario y una función para agregarle puntos **sin mutar** el original.

```
;; 1. Definimos nuestro estado inicial (immutable)
(def usuario {:nombre "Juan" :puntos 0})

;; 2. Creamos la funcion que "actualiza"
(defn agregar-puntos [user cantidad]
  (let [puntos-actuales (:puntos user)
        nuevos-puntos (+ puntos-actuales cantidad)]
    (assoc user :puntos nuevos-puntos)))

;; 3. Comprobamos la inmutabilidad en el REPL
user=> (agregar-puntos usuario 50)
{:nombre "Juan", :puntos 50}

user=> usuario
{:nombre "Juan", :puntos 0} ; El original sigue intacto!
```

Para la próxima sesión...

Exploraremos las **Funciones como Ciudadanos de Primera Clase**. Aprenderemos a tratar a las funciones como si fueran un dato más y dominaremos la trinidad de la programación funcional: **map, filter y reduce**.

- Crearemos y pasaremos funciones anónimas (`fn` o `#(...)`) como argumentos.
- Reemplazaremos por completo la necesidad de usar bucles tradicionales.

¡Sigán practicando en 4Clojure!